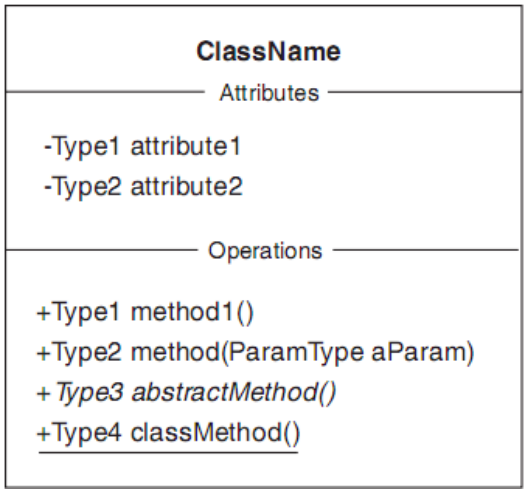
Modul 5 Relasi antar Kelas

Tujuan Praktikum
1. Praktikan dapat mengerti konsep dasar Object oriented programming 2. Praktikan dapat memahami penggunaan kelas dalam pemrograman java 3. Praktikan dapat memahami hubungan antar kelas baik asosiasi, agregasi, dan komposisi..

5.1 Diagram Kelas

Diagram kelas merupakan sebuah diagram yang **digunakan untuk memodelkan** kelas-kelas yang digunakan di dalam sistem beserta hubungan antar kelas dalam sistem tersebut.

Beberapa elemen penting dalam diagram kelas adalah kelas dan relasi antar kelas. Kelas digambarkan dengan simbol kotak seperti gambar di bawah ini:



Gambar 5-1 Diagram Kelas

Baris pertama dari simbol diagram kelas menandakan nama dari kelas yang bersangkutan. Baris di bawahnya menyatakan atribut-atribut dari kelas tersebut apa saja, dan baris setelahnya menyatakan *method-method* yang terdapat pada kelas tersebut. Adapun simbol untuk *access modifer* adalah:

Tabel 5-1 Simbol untuk akses modifier

Simbol	Keterangan
+	simbol *access modifier public*
-	simbol *access modifier private*
#	simbol *access modifier protected* (untuk simbol ini tidak terdapat pada gambar diatas)

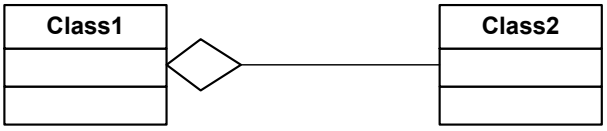
Sedangkan untuk menggambarkan **hubungan antar kelas** digunakan simbol garis antara dua kelas yang berelasi. Simbol garis tersebut antara lain:

a. Class1 **berasosiasi** dengan Class2 , digambarkan sebagai berikut:



Gambar 5-2 Hubungan Asosiasi.

b. Class2 merupakan elemen part-of dari Class1 (Class1 berelasi agregasi dengan Class2), digambarkan sebagai berikut:



Gambar 5-3 Hubungan Agregasi.

c. Class1 dengan Class2 berelasi komposisi, digambarkan sebagai berikut:



Gambar 5-4 Hubungan Komposisi

Fakultas Informatika
School of Computing
Telkom University
informatics lab

d. Class1 merupakan turunan dari Class2, digambarkan sebagai berikut:



Gambar 5-5 Hubungan Generalisasi

5.2 Hubungan Antar Kelas

Dalam *Object Oriented Programming*, kelas-kelas yang terbentuk dapat memiliki hubungan satu dengan yang lainnya, sesuai dengan kondisi dari kelas-kelas yang bersangkutan. Ada beberapa jenis hubungan yang dapat terjadi antara kelas yang satu dengan kelas yang lainnya, antara lain:

a. Asosiasi
b. Agregasi
c. Komposisi
d. Generalisasi dan Spesialisasi (terkait dengan pewarisan)

5.2.1 Asosiasi

Asosiasi merupakan hubungan antara dua kelas terpisah yang terbentuk melalui objek pada tiap kelas. Jika dua kelas dalam suatu model perlu berkomunikasi satu sama lain, harus ada hubungan di antara kelas hal tersebut dapat diwakili oleh **asosiasi**. Contoh hubungan asosiasi ini adalah:



Gambar 5-6 Contoh hubungan asosiasi.

Pada diagram kelas di atas, terlihat hubungan antara kelas Dosen dengan kelas Mahasiswa. Method pada Kelas Mahasiswa dipanggil pada kelas Mahasiswa sehingga memiliki hubungan asosiasi.

Implementasi dari diagram kelas tersebut dalam Java adalah sebagai berikut:

File Mahasiswa.java

```
public class Mahasiswa {
    private String nim;
    private String nama;

    public void setNama(String nama){
        this.nama = nama;
    }
    public void setNim(String nim){
        this.nim = nim;
    }
    public String getNama(){
        return this.nama;
    }


    public String getNim(){
        return this.nim;
    }
}
```

File Dosen.java

```
public class Dosen {
    private String kodeDosen;
    private String namaDosen;

    //Setter
    public void setKodeDosen(String kodeDosen){
        this.kodeDosen = kodeDosen;
    }
    public void setNamaDosen(String namaDosen){
        this.namaDosen = namaDosen;
    }
    //Getter
    public String getKodeDosen(){
```

```
        return this.kodeDosen;
    }
    public String getNamaDosen(){
        return this.namaDosen;
    }

    public void giveScore(Mahasiswa s, int nilai){
      // ini asosiasi, method milik class Student dipanggil di class
Dosen,
       // tp objek Student tidak menjadi atribut dr class Lecture

         s.setNilai(nilai);
    }
    public int getScore(Mahasiswa s){
      // ini asosiasi, method milik class Student dipanggil di class
Lecture,
      // tp objek Student tidak menjadi atribut dr class Lecture

         return s.getNilai();
    }

    public static void main(String[] args) {
        Mahasiswa m = new Mahasiswa();

        m.setNim("130118383");
        m.setNama("Budi");

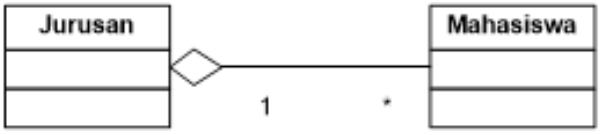
        Dosen d = new Dosen();
        d.giveScore(m, 90);

        System.out.println("Nim :"+m.getNim());
        System.out.println("Nama :"+m.getNama());
        System.out.println("Nilai :"+d.getScore(m));
    }
}
```

Pada implementasi di atas terlihat bahwa method pada kelas Mahasiswa dipanggil pada kelas Dosen pada method getScore() dan giveScore() sehingga method pada kelas Dosen menerima nilai dari objek kelas Mahasiswa.

5.2.2 Agregasi

Agregasi merupakan **hubungan** antara **dua kelas** yang **lebih ketat** dan terdapat hubungan **bagian** (*part*) dan **keseluruhan (*whole*)**. Contoh hubungan agregasi ini adalah:



Gambar 5-7 Contoh hubungan agregasi

Pada diagram kelas di atas, terlihat hubungan antara kelas Jurusan dengan kelas Mahasiswa. Kelas mahasiswa merupakan bagian dari kelas jurusan, tetapi kelas jurusan dan kelas mahasiswa dapat diciptakan sendiri-sendiri.

Implementasi dari diagram kelas tersebut dalam Java adalah sebagai berikut:

File Mahasiswa.java

```
public class Mahasiswa {
	private String NIM, Nama;
	public Mahasiswa(String no, String nm) {
		this.NIM = no;
		this.Nama = nm;
	}

	public String GetNIM() {
		return (NIM);
	}

	public String GetNama() {
		return (Nama);
	}
}
```

File Jurusan.java

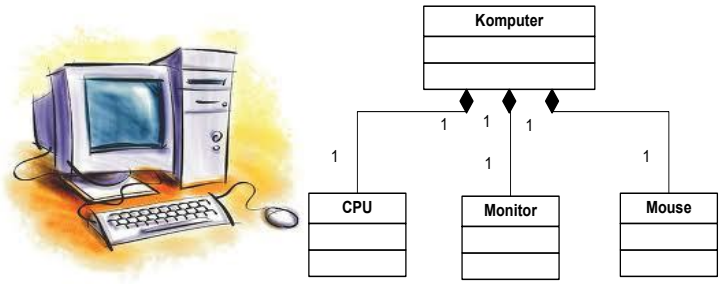
```
public class Jurusan {
	private String KodeJurusan, NamaJurusan;
	private Mahasiswa[] Daftar = new Mahasiswa[10];
	public Jurusan(String kode, String nama) {
		this.KodeJurusan = kode;
		this.NamaJurusan = nama;
	}

	private static int JmlMhs = 0;
	public void AddMahasiswa(Mahasiswa m) {
		this.Daftar[JmlMhs] = m;
		this.JmlMhs++;
	}
	public void DisplayMahasiswa() {
		int i;
		Sistem.out.println("Kode Jurusan : "+this.KodeJurusan);
		Sistem.out.println("Nama Jurusan : "+this.NamaJurusan);
		Sistem.out.println("Daftar Mahasiswa :");
		for (i=0;i<JmlMhs;i++)
		Sistem.out.println(Daftar[i].GetNIM()+"
	"+Daftar[i].GetNama());
	}
}
```

Pada implementasi terlihat bahwa kelas jurusan memiliki atribut yang memiliki tipe kelas mahasiswa, sehingga kelas mahasiswa merupakan bagian dari kelas jurusan.

5.2.3 Komposisi

Komposisi merupakan **bentuk khusus dari agregasi** dimana terdapat hubungan antar kelas yang lebih kuat, karena yang menjadi **part (bagian)** tidak akan ada tanpa adanya kelas yang menjadi whole. Contoh hubungan komposisi adalah sebagai berikut:



Gambar 5-8 Contoh hubungan komposisi.

Dari diagram kelas di atas terlihat bahwa kelas CPU, Monitor, dan Mouse semuanya merupakan bagian dari kelas Komputer dan ketika kelas Komputer musnah maka kelas CPU, *Monitor*, dan *Mouse* akan ikut musnah. Implementasi dari diagram kelas tersebut dalam Java adalah sebagai berikut:

File CPU.java

```
public class CPU {
	private String Merk;
	private int Kecepatan;
	public CPU(String m, int k) {
		this.Merk = m;
		this.Kecepatan = k;
	}
	public void DisplaySpecCPU() {
		Sistem.out.println(this.Merk + ", " + this.Kecepatan);
	}
}
```

File Monitor.java

```
public class Monitor {
	private String Merk;
	public Monitor(String m) {
		this.Merk = m;
	}

	public void DisplaySpecMonitor() {
		Sistem.out.println(this.Merk);
	}
}
```

File Mouse.java

```
public class Mouse {
	private String Merk, Type;
	public Mouse(String m, String t) {
		this.Merk = m;
		this.Type = t;
	}

	public void DisplaySpecMouse() {
		Sistem.out.println(this.Merk + ", " + this.Type);
	}
}
```

Pada implementasi di atas, terlihat bahwa kelas CPU, Monitor, dan Printer merupakan atribut dari kelas Komputer dan baru diciptakan pada saat instansiasi objek dari kelas Komputer.:

Fakultas Informatika
School of Computing
Telkom University
informatics lab